

Chunking Strategies in RAG

Advanced Data Preparation Techniques



Why Chunking Matters



Highest leverage decision
in RAG

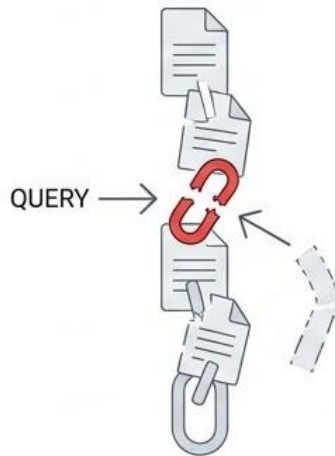


Poor chunking destroys
performance

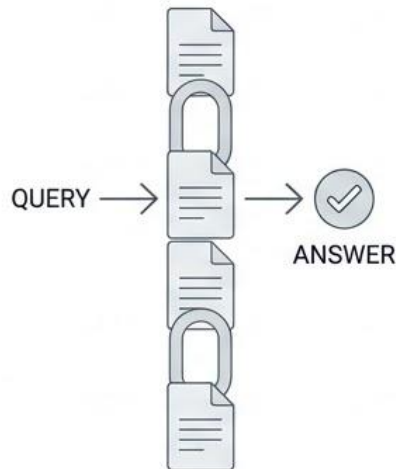


Affects accuracy, cost, and
context

BROKEN CHAIN
(FRAGILE)



STRONG CHAIN
(CONNECTED)



Fixed-Width Chunking

DEFINITION:

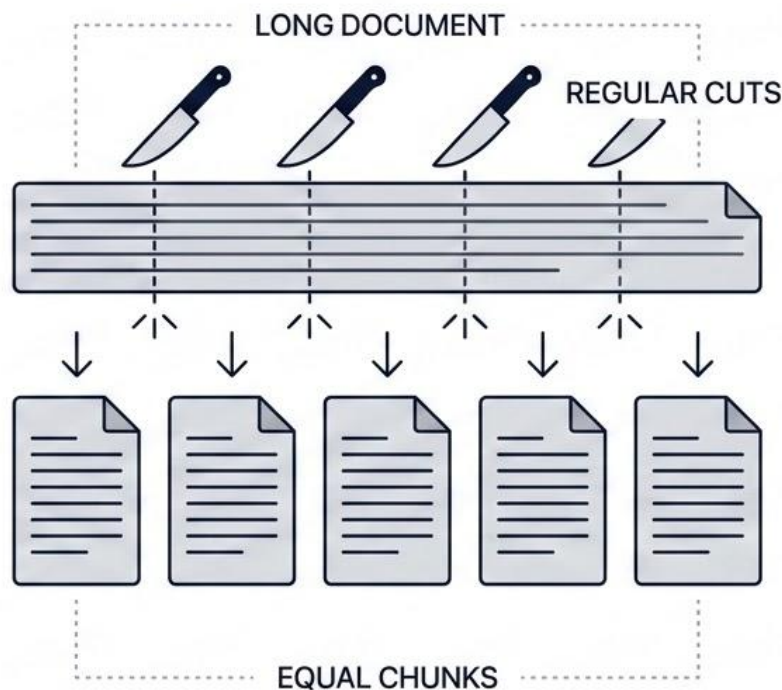
Simply splitting a document into chunks of a fixed, predetermined number of tokens, words, or characters, regardless of semantic content.

PROS:

- Predictable & Simple Implementation
- Low Computational Overhead

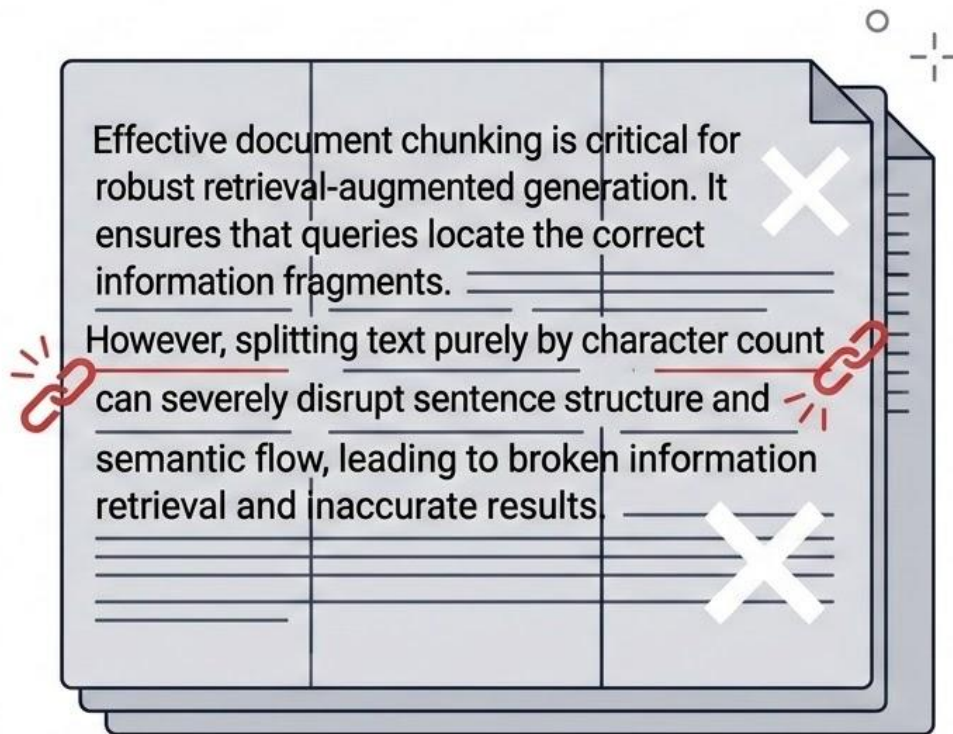
CONS:

- Arbitrary Breaks (e.g., in sentences, paragraphs)
- Can Sever Important Semantic Context
- Inefficient with Varied Content Structure



Why Fixed-Width Chunking Fails

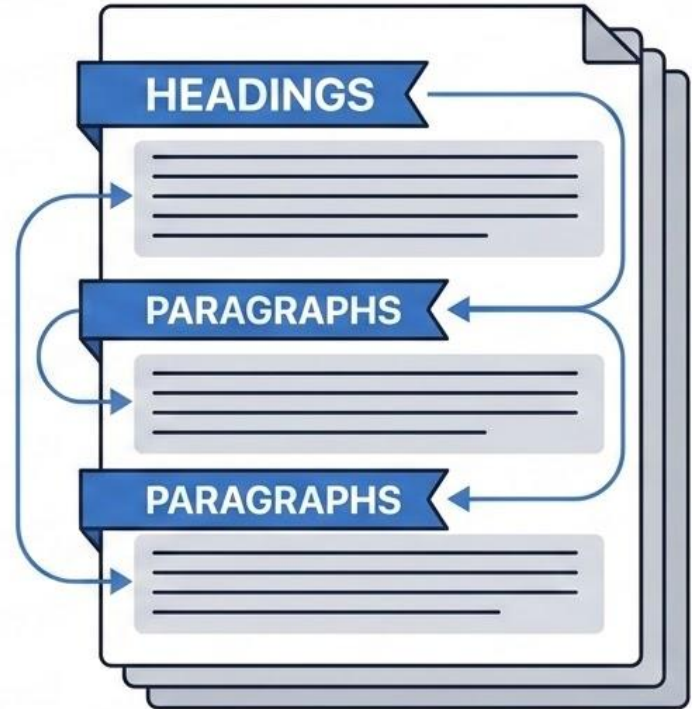
- ⚠ Arbitrary Breaks
- ⚠ Lost Semantic Context
- ⚠ Inefficient Structure
- ⚠ Unpredictable Query Matches



Semantic Chunking by Structure

Main idea: Respect natural boundaries

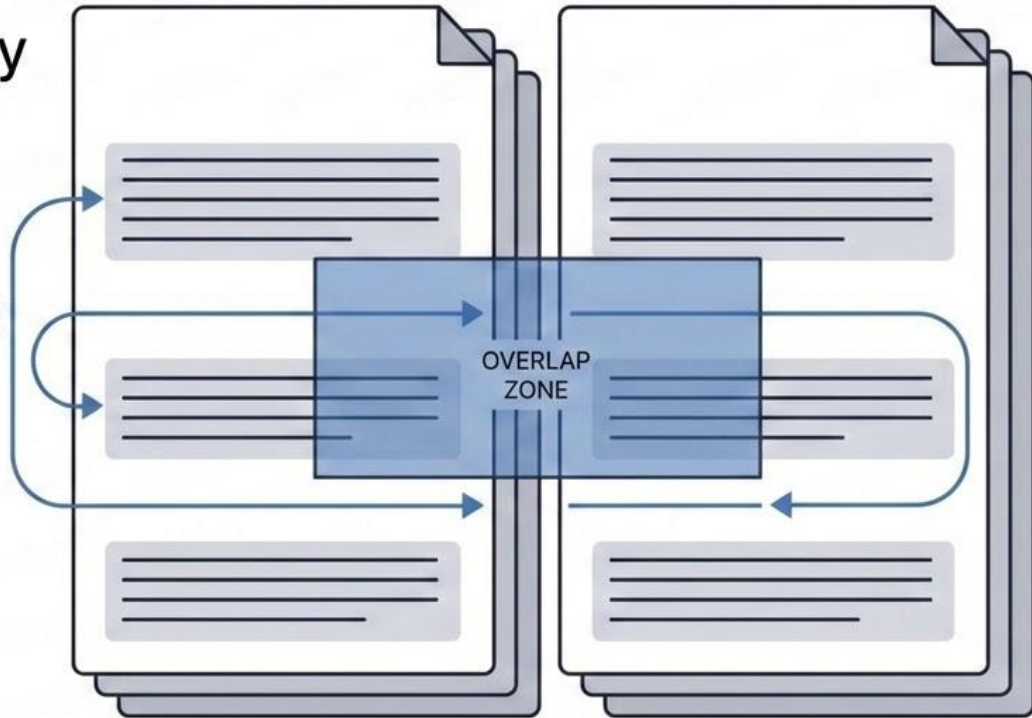
- Headings
- Paragraphs
- Logical Units



Overlap Windows

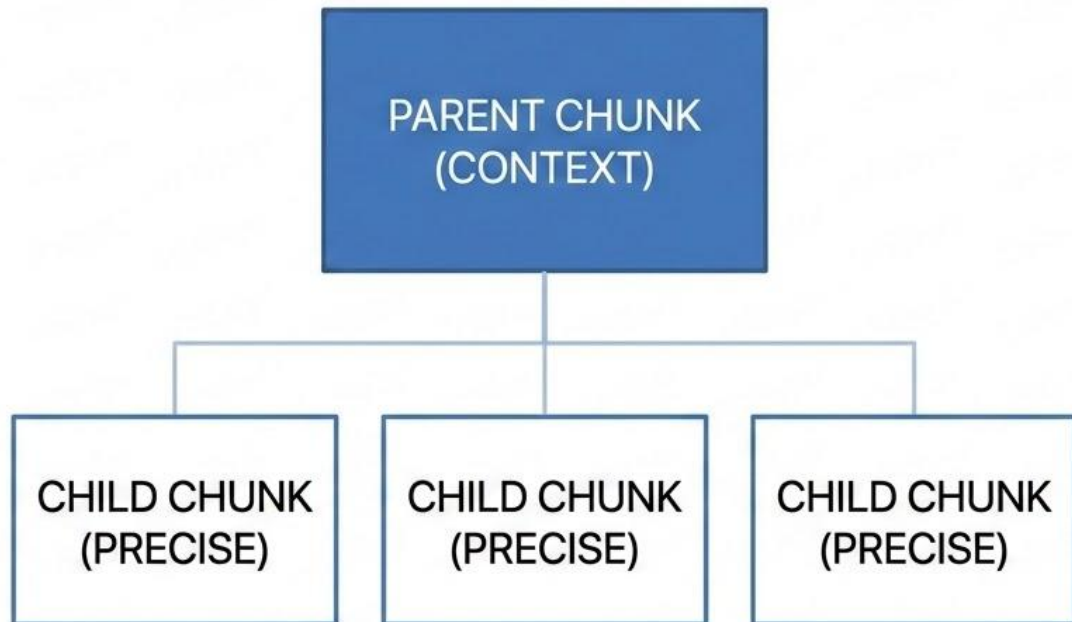
Purpose: Ensuring continuity and context flow between retrieved text chunks.

- Recommended Overlap: 10-20%



Parent-Child Chunking

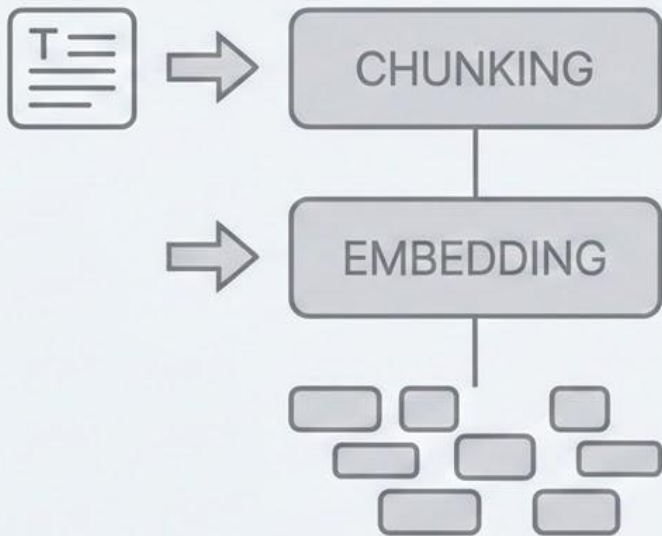
Concept: Small precise chunks + Large context chunks



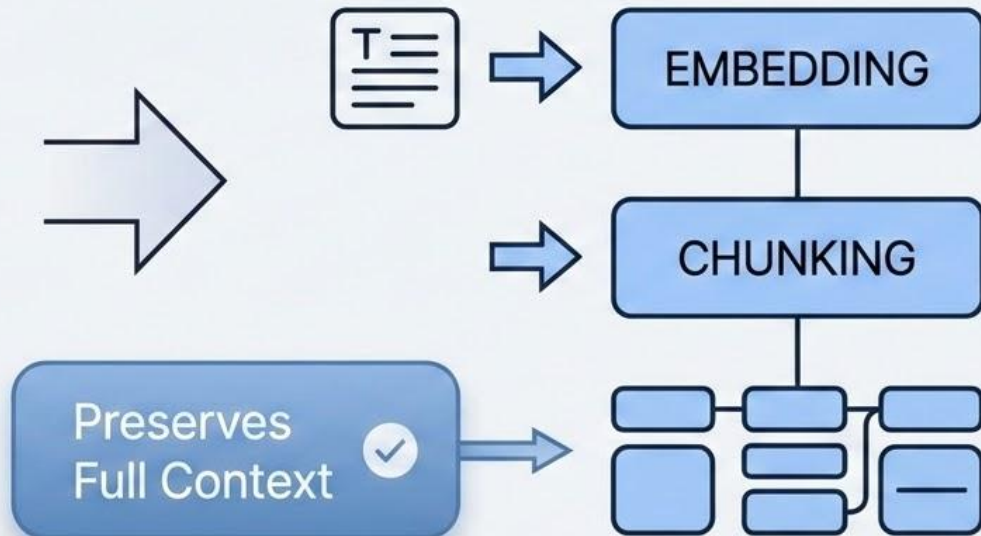
Late Chunking

Embed First → Chunk Later

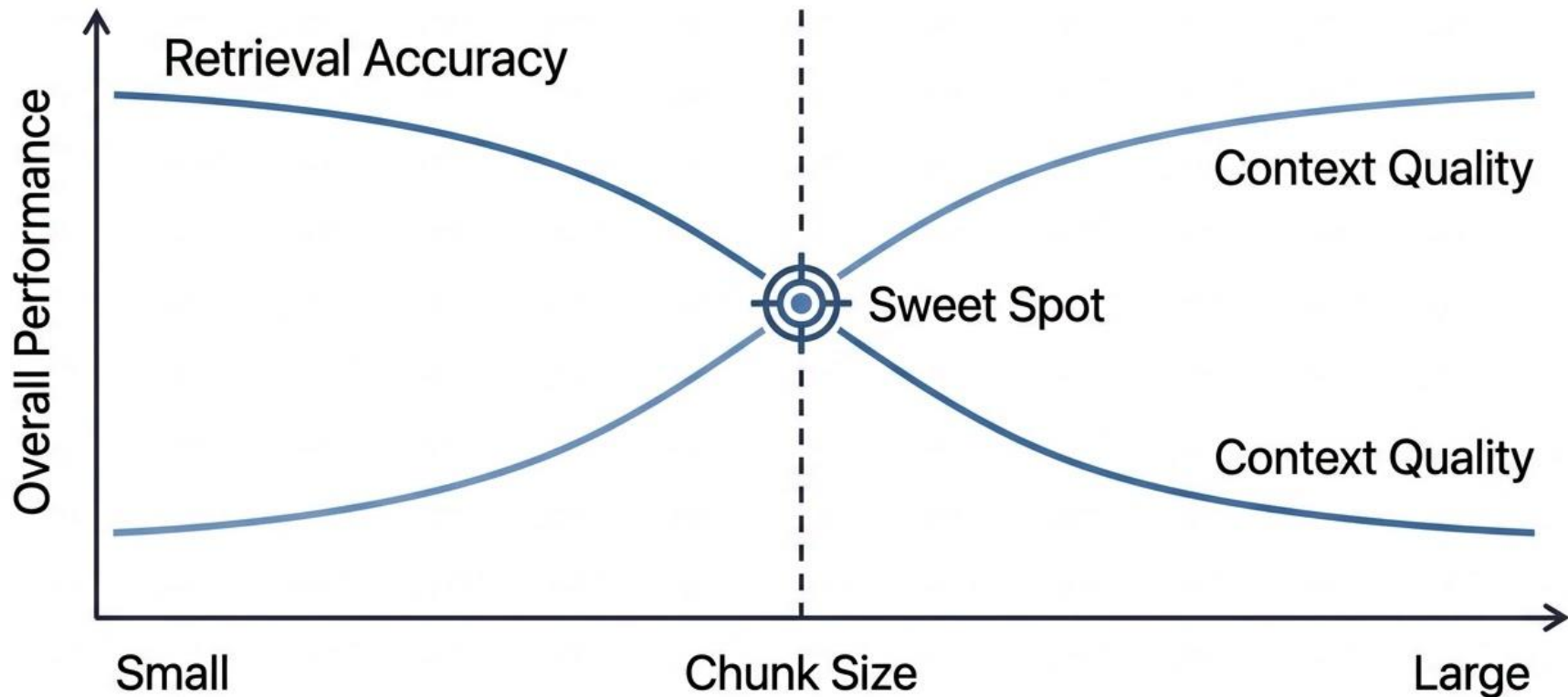
Traditional (Chunk → Embed)



Late Chunking



Chunk Size Tradeoff



Chunking Strategies Comparison

	Strategy	Speed	Semantic Quality	Context	Best For
	Fixed.	High	Low	Lost (Breaks Meaning)	Fast, brute-force processing.
	Semantic.	Medium	High (Preserves Local Meaning)	Moderate	General structured text documents.
	Overlap.	Medium	High	High (Links Boundaries)	Ensuring critical detail is not missed
	Parent-Child.	Low	Very High (Contextual)	Exceptional (Full Context Link)	Large, complex documents.
	Late Chunking.	Ultra-Fast (Pre-Calculated)	Exceptional	Maximum (Preserves Full Context)	High-performance, contextual search.

Best Practices

- ✓ 500-800 tokens for most cases
- ✓ Prefer semantic + overlap
- ✓ Combine multiple techniques
- ✓ Always test & measure
- ✓ Consider Late Chunking for critical apps

Key Takeaways

- ✓ • Chunking is critical for RAG success
- ✓ • Move beyond fixed-width
- ✓ • Use semantic and hierarchical methods
- ✓ • Late Chunking offers superior context



Great RAG starts with great chunking